

LAPORAN PRAKTIKUM PBO
PRAKTIKUM 4 Ekspansi Produk Keuangan (Inheritance)

Disusun oleh :

M. Fajar Fadhilul Zikri

NIM: 2511533023

Dosen Pengampu: **Nurfiah, S.ST, M.Kom.**

Asisten Praktikum: **Muhammad Dawi Syauqi**



FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
DEPARTEMEN INFORMATIKA
TAHUN 2026

A. Tahapan Pelaksanaan Praktikum

Pada praktikum ini, program sistem perbankan mini dari pekan sebelumnya disempurnakan melalui penerapan prinsip **Inheritance** (Pewarisan) serta pembuatan *subclass* baru, yaitu RekeningTabungan dan RekeningGiro. Selain itu, integrasi kelas Transaksi dan penerapan **Enkapsulasi** difokuskan untuk menjaga keamanan dan integritas data. Enkapsulasi merupakan prinsip dasar Pemrograman Berbasis Objek (OOP) yang menyembunyikan detail implementasi serta atribut sensitif suatu objek dari akses langsung pihak luar, sehingga data hanya dapat dibaca atau dimodifikasi melalui method resmi yang memiliki mekanisme validasi dan otentikasi.

1. Membuat Class Rekening

Langkah pertama class Rekening dari praktikum sebelumnya tanpa kode yang diubah. Kode programnya adalah sebagai berikut.

```
import java.util.ArrayList;

public class Rekening {
    // 1. Mengunci atribut menjadi private
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif
    private ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN saat Instansiasi / Constructor
        if (pinAwal != null && pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit!
Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() {
        return nomorRekening;
    }
}
```

```

    }

    public String getNamaPemilik() {
        return namaPemilik;
    }

    // 4. Method Otentikasi Internal PIN
    public boolean otentikasi(String inputPin) {
        return this.pin != null && this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            this.saldo += nominal;

            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            this.saldo -= nominal;
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cetakMutasi() {
        System.out.println("\n=== MUTASI REKENING ===");
        System.out.println("NO. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("-----");

        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi trx : riwayatTransaksi) {
                trx.cetakDetail();
            }
        }
    }

```

```

    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("NO. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}
}

```

Penjelasan: Class Rekening berfungsi sebagai cetak biru (*blueprint*) untuk memodelkan data rekening bank beserta operasinya secara aman menggunakan prinsip enkapsulasi. Class ini menyimpan lima atribut utama, yaitu nomorRekening, namaPemilik, saldo, pin, dan riwayatTransaksi. Constructor digunakan untuk menginisialisasi data awal sekaligus memvalidasi PIN (wajib 6 digit, atau diganti PIN *default*). Method otentikasi() digunakan untuk verifikasi PIN, sedangkan setorTunai() dan tarikTunai() memproses transaksi dengan validasi nominal dan kecukupan saldo serta mencatatnya ke histori. Selain itu, terdapat method cetakMutasi() untuk melihat riwayat transaksi, cekInformasi() untuk menampilkan detail akun dan saldo akhir, serta beberapa method *getter* untuk mengakses data publik.

Untuk praktikum ini kita akan mengubah private ke protected:

```

protected double saldo;
protected ArrayList<Transaksi> riwayatTransaksi;

```

tujuan nya Agar kelas anak (Tabungan, Giro) bisa memanipulasi saldo dan mencatat riwayatTransaksi tanpa harus membuka akses tersebut ke publik, atribut tersebut harus menggunakan modifier protected.

2. Membuat subclass RekeningTabungan:

```

public class RekeningTabungan extends Rekening {
    // Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    // Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal,
        String pinAwal, double sukuBunga) {

        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }
}

```

```
// Method untuk menambahkan bunga pada akhir bulan
public void tambahBungaAkhirBulan() {

    double nominalBunga = saldo * (sukuBunga / 100);
    saldo += nominalBunga;

    // Mencatat riwayat transaksi
    String idTrx = "TRX-B-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Bunga",
nominalBunga));

    System.out.println("Bunga " + sukuBunga + "% berhasil
ditambahkan: Rp " + nominalBunga);
}
}
```

Class RekeningTabungan merupakan kelas anak yang mewarisi seluruh atribut dan metode dari kelas induk Rekening dengan menggunakan kata kunci extends. Kelas ini memiliki atribut spesifik sukuBunga serta konstruktor yang memanfaatkan pemanggilan super() untuk menginisialisasi data dasar rekening pada kelas induk. Di dalamnya juga terdapat metode tambahBungaAkhirBulan() yang berfungsi menghitung nominal bunga berdasarkan persentase sukuBunga, menambahkan nominal tersebut secara langsung ke variabel saldo, serta mencatat riwayat transaksi barunya ke dalam koleksi riwayatTransaksi.

3. Membuat Subclass RekeningGiro:

```
public class RekeningGiro extends Rekening {
    // Atribut spesifik yang hanya dimiliki oleh Giro
    private double batasOverdraft;

    // Constructor Subclass
    public RekeningGiro(String nomor, String nama, double saldoAwal,
String pinAwal, double batasOverdraft) {
        // Memanggil konstruktor kelas induk (Rekening)
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter untuk batas overdraft
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

Class RekeningGiro merupakan subclass yang mewarisi atribut dan metode dasar dari class induk Rekening dengan menggunakan kata kunci extends. Class ini dilengkapi dengan atribut spesifik batasOverdraft untuk menyimpan limit fasilitas pinjaman nasabah, serta konstruktor yang memanggil super() di baris pertama untuk

menginisialisasi identitas rekening pada class induk. Selain itu, terdapat method *getter* `getBatasOverdraft()` yang digunakan untuk mengakses nilai batas *overdraft* secara aman karena atributnya dideklarasikan dengan akses *private*.

4. Membuat Class Main

Langkah selanjutnya class Main sebagai program utama (driver) dari praktikum sebelumnya tanpa kode yang diubah. Kode programnya adalah sebagai berikut.

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main1 {
    private static Rekening akunAktif;

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\n-----");
            if (akunAktif != null) {
                System.out.println("Akun Aktif: " +
                    akunAktif.getNamaPemilik() + " (" + akunAktif.getNomorRekening() + ")");
            } else {
                System.out.println("Akun Aktif: Belum Ada");
            }
            System.out.println("-----");
            System.out.println("Menu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("7. Keluar");
            System.out.print("Pilih Menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No rekening: ");
                    String no = input.nextLine();
```

```

        System.out.print("Masukkan Nama Pemilik: ");
        String nama = input.nextLine();
        System.out.print("Masukkan Saldo Awal: ");
        double saldo = input.nextDouble();
        input.nextLine();

        // Meminta input PIN 6 digit
        System.out.print("Masukkan PIN (6 digit): ");
        String pin = input.nextLine();

        Rekening akunBaru = new Rekening(no, nama,
saldo, pin);

        daftarRekening.add(akunBaru);

        akunAktif = akunBaru;
        break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error: Mohon maaf,
anda belum memilih/memiliki rekening!");
        } else {
            System.out.print("Masukkan nominal
setor: ");

            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Mohon maaf,
anda belum memilih/memiliki rekening!");
        } else {
            // Minta PIN sebelum Tarik Tunai
            System.out.print("Masukkan PIN Anda:
");

            String inputPinTarik =
input.nextLine();

            if
(akunAktif.otentikasi(inputPinTarik)) {
                System.out.print("Masukkan
nominal tarik: ");

                double tarik =
input.nextDouble();

                akunAktif.tarikTunai(tarik);
            } else {
                System.out.println("Akses
Ditolak: PIN yang Anda masukkan salah!");
            }
        }
        break;

    case 4:
        if (akunAktif == null) {

```

```

        System.out.println("Error: Anda belum
memilih/membuka rekening!");
    } else {
        akunAktif.cekInformasi();
    }
    break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening
yang terdaftar di dalam sistem.");
        } else {
            System.out.print("Masukkan No Rekening
yang dicari: ");

            String noCari = input.nextLine();
            boolean ditemukan = false;

            for (Rekening rek : daftarRekening) {
                if
(rek.getNomorRekening().equalsIgnoreCase(noCari)) {
                    akunAktif = rek;
                    ditemukan = true;
                    // Menggunakan getter
getNamaPemilik()

                    System.out.println("Berhasil beralih ke rekening atas nama: " +
akunAktif.getNamaPemilik());

                    break;
                }
            }

            if (!ditemukan) {
                System.out.println("Error: Nomor
rekening tidak ditemukan!");
            }
        }
        break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum
memilih/membuka rekening!");
        } else {
            // Minta PIN sebelum Cetak Mutasi
            System.out.print("Masukkan PIN Anda:
");

            String inputPinMutasi =
input.nextLine();

            if
(akunAktif.otentikasi(inputPinMutasi)) {
                akunAktif.cetakMutasi();
            } else {
                System.out.println("Akses
Ditolak: PIN yang Anda masukkan salah!");
            }
        }
    }
}

```

```

        break;

        case 7:
            isRunning = false;
            System.out.println("Sistem ditutup. Terima
kasih!");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
        }
    }
    input.close();
}
}

```

Penjelasan: Class Main1 berfungsi sebagai antarmuka utama (program *driver*) bertipe teks untuk menjalankan sistem perbankan mini secara interaktif melalui konsol. Class ini mengelola daftar rekening terdaftar menggunakan ArrayList serta menyimpan referensi akunAktif yang sedang digunakan. Program menyediakan perulangan menu utama yang memungkinkan pengguna untuk membuka rekening baru, melakukan setor tunai, serta melakukan penarikan tunai dan cetak mutasi yang keduanya dilindungi otentikasi PIN. Selain itu, pengguna dapat melihat informasi rekening, berganti akun aktif berdasarkan pencarian nomor rekening, dan menghentikan jalannya sistem melalui pilihan menu keluar.

5. Membuat Class Transaksi

Langkah berikutnya adalah menambahkan class baru bernama Transaksi dari praktikum sebelumnya tanpa kode yang diubah. Kode programnya adalah sebagai berikut.

```

public class Transaksi {
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // konstruktor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // getter (Read-Only)
    public String getIdTransaksi() {return idTransaksi; }
    public String getJenis() { return jenis; }
}

```

```
        public double getNominal() { return nominal; }

        public void cetakDetail() {
            System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
" | Nominal: Rp" + nominal);
        }
    }
}
```

Penjelasan: Class Transaksi berfungsi sebagai cetak biru (blueprint) untuk memodelkan data entitas riwayat transaksi pada sistem perbankan. Class ini mengamankan tiga atribut utamanya, yaitu idTransaksi, jenis, dan nominal dengan visibilitas *private*. Constructor digunakan untuk menginisialisasi nilai awal seluruh atribut saat transaksi baru dibuat, sedangkan beberapa method *getter* disediakan sebagai akses *read-only* untuk membaca data tanpa dapat mengubahnya. Selain itu, terdapat method cetakDetail() yang berfungsi untuk menampilkan rincian informasi transaksi secara rapi ke konsol.

B. Latihan/Tugas

Sekarang sistem telah memiliki berbagai bentuk rekening. Silahkan memperbarui antarmuka CLI di Main.java agar mendukung produk-produk baru ini secara interaktif.

Tugas 1: Pembuatan Multi-Produk (40%)

Modifikasi case 1: Buka Rekening Baru di Main.java.

1. Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
2. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
3. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
4. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Tugas 2: Simulasi Waktu Akhir Bulan : 60%)

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
2. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

A. Ada Tambahan Fitur pada pilihan menu:

```
System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
System.out.println("0. Keluar");
System.out.print("Pilih Menu: ");
```

1. Tugas 1 Pembuatan Multi-Produk

```
case 1:
    System.out.print("Masukkan Nomor Rekening : ");
    String nomor = input.nextLine();
    System.out.print("Masukkan Nama Pemilik : ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal : ");
    double saldoAwal = input.nextDouble();
    input.nextLine();
```

```

        System.out.print("Masukkan PIN : ");
        String pin = input.nextLine();

        // Pilihan jenis produk rekening
        System.out.println("Pilih Produk:");
        System.out.println("1. Tabungan Umum");
        System.out.println("2. Giro Bisnis");
        System.out.print("Pilihan (1/2): ");
        int jenisProduk = input.nextInt();
        input.nextLine();

        Rekening akunBaru = null;

        if (jenisProduk == 1) {
            // Input Suku Bunga untuk Rekening Tabungan
            System.out.print("Masukkan Suku Bunga (%) : ");
            double sukuBunga = input.nextDouble();
            input.nextLine();

            // Instansiasi objek RekeningTabungan
            akunBaru = new RekeningTabungan(nomor, nama,
            saldoAwal, pin, sukuBunga);
            System.out.println("Rekening Tabungan berhasil
            dibuat!");

        } else if (jenisProduk == 2) {
            // Input Batas Overdraft untuk Rekening Giro
            System.out.print("Masukkan Batas Overdraft : ");
            double batasOverdraft = input.nextDouble();
            input.nextLine();

            // Instansiasi objek RekeningGiro
            akunBaru = new RekeningGiro(nomor, nama,
            saldoAwal, pin, batasOverdraft);
            System.out.println("Rekening Giro berhasil
            dibuat!");

        } else {
            System.out.println("Pilihan produk tidak valid!
            Pendaftaran dibatalkan.");
            break;
        }

        // Upcasting: Menyimpan objek Subclass ke
        daftarRekening bertipe Rekening
        daftarRekening.add(akunBaru);
        akunAktif = akunBaru;
        break;

```

Penjelasan: case 1: tersebut berfungsi untuk memproses pendaftaran rekening baru secara interaktif, di mana pengguna diminta menginput data umum (nomor rekening, nama, saldo awal, dan PIN) serta memilih jenis produk yang ingin dibuat (Tabungan Umum atau Giro Bisnis). Berdasarkan pilihan tersebut, sistem meminta input spesifik (suku bunga untuk Tabungan atau batas *overdraft* untuk Giro) lalu

membuat objek dari kelas anaknya (RekeningTabungan atau RekeningGiro). Terakhir, objek baru disimpan ke dalam koleksi daftarRekening melalui proses *Upcasting* dan langsung diset menjadi akun yang sedang aktif (akunAktif).

2. Tugas 2 Simulasi Waktu akhir Bulan

```
case 7:
    System.out.println("\n--- Simulasi Akhir Bulan ---");
    if (akunAktif == null) {
        System.out.println("Gagal: Silakan pilih / login
ke rekening terlebih dahulu!");
        break;
    }

    // Pengecekan tipe objek menggunakan instanceof
    if (akunAktif instanceof RekeningTabungan) {
        // Downcasting eksplisit ke tipe RekeningTabungan
        RekeningTabungan tabungan = (RekeningTabungan)
akunAktif;

        // Memanggil method spesifik milik
RekeningTabungan
        tabungan.tambahBungaAkhirBulan();
    } else {
        // Pesan penolakan jika akunAktif bukan
RekeningTabungan
        System.out.println("Gagal: Fitur bunga akhir
bulan hanya berlaku untuk Rekening Tabungan.");
    }
    break;
```

Penjelasan: case 7: tersebut berfungsi untuk menjalankan fitur simulasi bunga akhir bulan dengan memastikan terlebih dahulu bahwa ada akun yang sedang aktif/terpilih. Jika akun aktif ditemukan, sistem menggunakan operator instanceof untuk mengecek apakah objek akunAktif tersebut merupakan instance dari kelas RekeningTabungan. Jika benar, sistem melakukan *downcasting* eksplisit dari tipe Rekening ke RekeningTabungan agar dapat memanggil method spesifik tambahBungaAkhirBulan(). Sebaliknya, jika akun aktif bertipe lain (seperti RekeningGiro), sistem akan menolak transaksi dan menampilkan pesan bahwa fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

6. Hasil Output Program

```
=== SISTEM PERBANKAN MINI ===

-----
Akun Aktif: Belum Ada
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 1
Masukkan Nomor Rekening : 12345
Masukkan Nama Pemilik   : jar
Masukkan Saldo Awal    : 500000
Masukkan PIN           : 098765
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan (1/2): 1
Masukkan Suku Bunga (%) : 5
Rekening atas nama jar berhasil dibuat dengan saldo Rp500000.0
Rekening Tabungan berhasil dibuat!

-----
Akun Aktif: jar (12345)
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 7

--- Simulasi Akhir Bulan ---
Bunga 5.0% berhasil ditambahkan: Rp 25000.0

-----
Akun Aktif: jar (12345)
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
```

```
0. Keluar
Pilih Menu: 6
Masukkan PIN Anda: 098765

=== MUTASI REKENING ===
NO. Rekening : 12345
Nama Pemilik : jar
-----
ID: TRX-B-1791386789093 | Jenis: Bunga | Nominal: Rp25000.0

-----
Akun Aktif: jar (12345)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 1
Masukkan Nomor Rekening : 09876
Masukkan Nama Pemilik : iyan
Masukkan Saldo Awal : 700000
Masukkan PIN : 876543
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan (1/2): 2
Masukkan Batas Overdraft : 200000
Rekening atas nama iyan berhasil dibuat dengan saldo Rp700000.0
Rekening Giro berhasil dibuat!

-----
Akun Aktif: iyan (09876)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 7

--- Simulasi Akhir Bulan ---
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

-----
Akun Aktif: iyan (09876)
-----
Menu Utama:
1. Buka Rekening Baru
```

```
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 0
Sistem ditutup. Terima kasih!
```

Penjelasan: hasil output menunjukkan bahwa fitur multi produk pada menu 1 berfungsi dengan baik karena setiap kita memilih menu 1 kita akan diminta untuk memilih apakah Tabungan Umum atau Giro Bisnis, jika memilih Tabungan umum maka akan diminta input bunga kalau pilih Giro Bisnis nanti di minta meninputkan batas overdraft lalu untuk pilihan menu 7 yang di tambahkan itu khusus untuk Tabungan Umum jika kita pilih Giro bisnis dan memilih menu 7 maka gagal dieksekusi dan ada pengumuman fitur ini hanya untuk Tabungan Umum, jika kita milih Tabungan Umum maka akan tampil berapa Bunga yang kita dapatkan.